

# Flash Read and Write Experiment

## Flash Read and Write Experiment

- 1. Flash Read and Write Overview
- 2. Core Concepts
  - 2.1 NVS and Preferences
  - 2.2 NVS Supported Data Types
  - 2.3 Namespaces and Key-Value Pairs
  - 2.4 Delete with Shift-Forward Algorithm
- 3. Hardware Dependencies
- 4. Project Architecture and Flow
- 5. Source Code
  - 5.1 Initialization and Boot Read
  - 5.2 Append Write
  - 5.3 Delete with Shift-Forward Algorithm
  - 5.4 Traverse Read
  - 5.5 Clear All Data
- 6. Operation Flow
  - 6.1 Compile and Flash
  - 6.2 Normal Phenomena
  - 6.3 Verification Methods
- 7. Common Issues

## 1. Flash Read and Write Overview

ESP32's internal SPI Flash can not only store firmware but also be partitioned for user data storage. This experiment uses Arduino's **Preferences library** (a high-level wrapper for ESP-IDF NVS) to implement CRUD operations on key-value pairs: append write, traverse read, delete by index (with shift-forward algorithm), and one-click clear.

**NVS (Non-Volatile Storage)** is a key-value storage library provided by ESP-IDF, based on a dedicated partition of SPI Flash. NVS encapsulates wear-leveling and power-failure protection on Flash, allowing developers to read and write persistent data like using a "dictionary" without worrying about the underlying erase/write details.

### Typical Application Scenarios:

| Application Type      | Examples                                                   |
|-----------------------|------------------------------------------------------------|
| Configuration Storage | Wi-Fi SSID/password, device name, calibration parameters   |
| Status Recording      | Device running time, last shutdown state, error log count  |
| User Data             | Preset parameter tables, historical records, offline cache |
| Firmware Management   | OTA upgrade flags, firmware version number                 |

## 2. Core Concepts

### 2.1 NVS and Preferences

NVS (Non-Volatile Storage) is a lightweight key-value storage library implemented by ESP-IDF on SPI Flash. Preferences is Arduino's wrapper for it:

```
Arduino Preferences → ESP-IDF NVS → SPI Flash partition → wear-leveling (WL) protection
```

### 2.2 NVS Supported Data Types

| Type        | Functions                                                                                                        | Size                   |
|-------------|------------------------------------------------------------------------------------------------------------------|------------------------|
| Integer     | <code>nvs_set_i8()</code> / <code>nvs_set_i16()</code> / <code>nvs_set_i32()</code> / <code>nvs_set_i64()</code> | 1~8 bytes              |
| String      | <code>nvs_set_str()</code>                                                                                       | Variable, ≤ 1984 bytes |
| Binary Blob | <code>nvs_set_blob()</code>                                                                                      | Variable, ≤ 1984 bytes |

### 2.3 Namespaces and Key-Value Pairs

- **Namespace:** Equivalent to a folder; data in different namespaces does not interfere with each other. This experiment uses `"data_store"`
- **Key-Value Pair:** Each storage item consists of a key (string) and a value (int/float/string/bytes)
- `begin(namespace, readonly)` mount → operate → `end()` commit and release

### 2.4 Delete with Shift-Forward Algorithm

NVS itself only supports `remove(key)` deletion by key and cannot automatically "fill gaps". This experiment implements a shift-forward strategy: suppose `dat3` is deleted, `dat4`→`dat3`, `dat5`→`dat4`... finally delete the extra `datN`, decrement counter by 1, ensuring key names are always consecutively numbered from 1.

## 3. Hardware Dependencies

Pure software experiment, no external hardware required. Only USB cable needed to view serial interaction menu.

#### ESP32-S3 NVS Hardware Features:

| Feature           | Parameter                                                         |
|-------------------|-------------------------------------------------------------------|
| Storage medium    | Internal SPI Flash (partitioned via partition table)              |
| Maximum key count | Limited by Flash partition size (typically hundreds to thousands) |

| Feature                  | Parameter                                                                                                           |
|--------------------------|---------------------------------------------------------------------------------------------------------------------|
| Key name length          | Up to 15 characters                                                                                                 |
| Value types              | int8/16/32/64, float/double, string, blob (binary)                                                                  |
| Maximum string length    | 4000 bytes (limited by NVS page size)                                                                               |
| Erase/write cycles       | About 100,000 times/sector (wear-leveling algorithm distributes writes evenly)                                      |
| Power-failure protection | Yes ( <code>end()</code> calls <code>nvs_commit</code> for atomic write, power failure won't corrupt existing data) |
| Write granularity        | Cached per namespace, <code>end()</code> commits all at once                                                        |

## 4. Project Architecture and Flow

```

setup()
├ Serial.begin(115200)
├ Read saved dataCount in read-only mode
└ Print interaction menu (1=Write 2=Read 3=Delete 4=Clear)

loop()
└ if Serial.available():
    ├── cmd=1 → wait for user input → putString("datN", content) → dataCount++
    ├── cmd=2 → Iterate i=1..count → getString("datN") → Print
    ├── cmd=3 → Shift-forward algorithm → remove(datN) → dataCount--
    └ cmd=4 → clear() → dataCount=0

```

## 5. Source Code

**Source Code Location:** `Source code` → `Arduino` → `2.Serial communication` → `4.FLASH` → `4.FLASH.ino`

### 5.1 Initialization and Boot Read

Read-only mode quickly reads the saved entry count without holding a write lock.

```

#include <Preferences.h>
Preferences prefs;
#define STORAGE_NAMESPACE "data_store"
int dataCount = 0;

void setup() {
    Serial.begin(115200);

    prefs.begin(STORAGE_NAMESPACE, true);    // Read-only mode
    dataCount = prefs.getInt("count", 0);    // Returns default 0 if key doesn't exist

```

```

prefs.end();

Serial.println("1=Write 2=Read 3=Delete 4=Clear");
}

```

The second parameter `0` in `getInt("count", 0)` is the default value — returns 0 instead of error when the key doesn't exist.

## 5.2 Append Write

Open in read-write mode, generate key name `datN`, write content and update counter.

```

if (cmd == "1") {
    // wait for user input content...
    prefs.begin(STORAGE_NAMESPACE, false);    // Read-write mode
    dataCount++;
    char key[16];
    sprintf(key, "dat%d", dataCount);        // Key name: "dat1", "dat2"...
    prefs.putString(key, content);
    prefs.putInt("count", dataCount);        // Update counter
    prefs.end();                            // Commit write
}

```

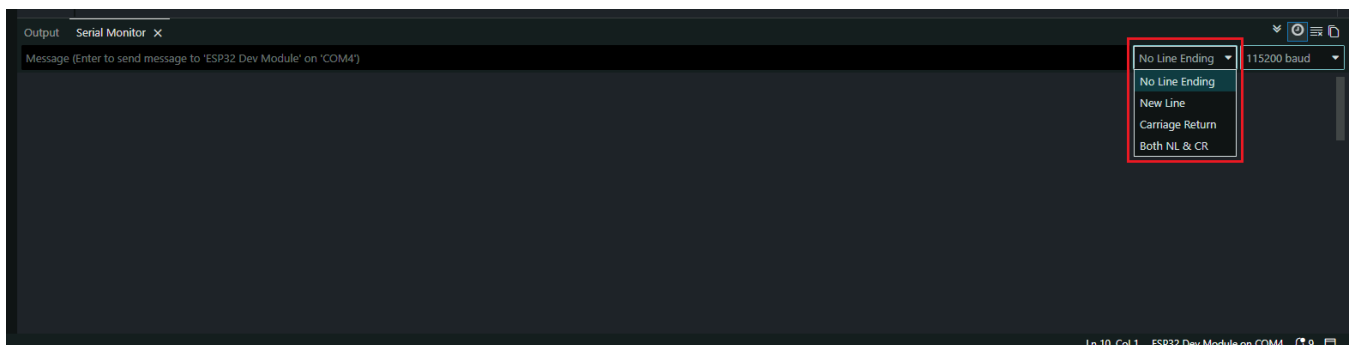
`prefs.end()` actually commits the write to Flash. Power failure before `end()` loses this write but doesn't damage existing data.

Important note for append write:

Serial monitor send settings must select **No line ending**.

If newline or carriage return is used as message terminator, pressing enter automatically appends `\r\n` characters;

If only enter is pressed without valid text input, the program reads empty content containing only newline characters, ultimately writing invalid empty data to Preferences.



## 5.3 Delete with Shift-Forward Algorithm

After deleting `datDe1Num`, shift subsequent data forward to maintain consecutive key names.

```

for (int i = delNum; i < total; i++) {
    char srcKey[16], dstKey[16];
    sprintf(srcKey, "dat%d", i + 1);           // Source key (next one)
    sprintf(dstKey, "dat%d", i);               // Destination key (current position)
    String nextData = prefs.getString(srcKey, "");
    prefs.putString(dstKey, nextData);         // Shift forward
}
sprintf(key, "dat%d", total);
prefs.remove(key);                            // Delete the extra last entry
dataCount = total - 1;
prefs.putInt("count", dataCount);

```

Why is the shift-forward algorithm needed? `prefs.remove(key)` only deletes a single entry. After deletion, key names will have gaps (e.g., `dat1`→`dat2`→`dat4`). If subsequent append writes only rely on `dataCount` auto-increment, `dat3`'s position will always be empty. The shift-forward algorithm ensures consecutive key names and, combined with append write logic, prevents "ghost holes".

## 5.4 Traverse Read

Open in read-only mode, loop through all `datN` keys and print. The second parameter `""` in `getString(key, "")` is the default value when the key doesn't exist.

```

else if (cmd == "2") {
    prefs.begin(STORAGE_NAMESPACE, true);      // Read-only mode
    int total = prefs.getInt("count", 0);

    if (total == 0) {
        Serial.println("No data");
    } else {
        for (int i = 1; i <= total; i++) {
            char key[16];
            sprintf(key, "dat%d", i);           // Construct key name "dat1", "dat2"...
            String data = prefs.getString(key, ""); // Returns empty string "" if not exists
            Serial.printf("Entry %d: %s\r\n", i, data.c_str());
        }
    }
    prefs.end();
}

```

Call `prefs.end()` for each read operation as well. Although read-only mode doesn't hold a write lock, timely resource release is good programming practice.

## 5.5 Clear All Data

`prefs.clear()` clears all key-value pairs in the current namespace at once, much faster than `remove()` one by one.

```

else if (cmd == "4") {
  prefs.begin(STORAGE_NAMESPACE, false);           // Read-write mode
  prefs.clear();                                   // Clear all key-value pairs in this
namespace
  dataCount = 0;                                   // Reset counter to zero
  prefs.end();
  serial.println("All data cleared!");
}

```

`clear()` only clears the current namespace `"data_store"` and does not affect other namespaces. The underlying operation marks old NVS pages as erased and allocates new pages — the entire namespace's data is one batch operation, more than 10 times faster than `remove()` one by one. **Note** don't forget to synchronously reset `dataCount` in memory, otherwise the counter will be inconsistent with actual storage.

## 6. Operation Flow

### 6.1 Compile and Flash

**Arduino Flash Process:** See 1. Before Development → 2. Build and Flash.

1. Open `4.FLASH.ino`
2. Open **Serial Monitor** (115200 bps), set line ending to "Newline"

### 6.2 Normal Phenomena

Interact via commands in the serial monitor:

```

=====
ESP32-S3 Flash Storage
1 = Append Write (追加写入)
2 = Read All Data (读取全部)
3 = Delete Specified Data (删除指定)
4 = Clear All Data (清空全部)
=====
Enter content to save:
Entry 1 saved successfully
Enter content to save:
Entry 2 saved successfully
All data:
Entry 1: Hello world!
Entry 2: ESP32-S3 NVS Demo
Enter entry number to delete:
Entry 1 deleted
All data:
Entry 1: ESP32-S3 NVS Demo
All data cleared!
No data

```

```

=====
ESP32-S3 Flash Storage
1 = Append Write (追加写入)
2 = Read All Data (读取全部)
3 = Delete Specified Data (删除指定)
4 = Clear All Data (清空全部)
=====
Enter content to save:
Entry 1 saved successfully
Enter content to save:
Entry 2 saved successfully
All data:
Entry 1: Hello World!
Entry 2: ESP32-S3 NVS Demo
Enter entry number to delete:
Entry 1 deleted
All data:
Entry 1: ESP32-S3 NVS Demo
All data cleared!
No data

```

Before power off:

```

=====
ESP32-S3 Flash Storage
1 = Append Write (追加写入)
2 = Read All Data (读取全部)
3 = Delete Specified Data (删除指定)
4 = Clear All Data (清空全部)
=====
Enter content to save:
Entry 1 saved successfully
Enter content to save:
Entry 2 saved successfully
All data:
Entry 1: Hello World!
Entry 2: ESP32-S3 NVS Demo

```

After power cycle reconnect:

```

=====
ESP32-S3 Flash Storage
1 = Append Write (追加写入)
2 = Read All Data (读取全部)
3 = Delete Specified Data (删除指定)
4 = Clear All Data (清空全部)
=====
All data:
Entry 1: Hello World!
Entry 2: ESP32-S3 NVS Demo

```

## 6.3 Verification Methods

| Operation                         | Expected Result                         |
|-----------------------------------|-----------------------------------------|
| Write, power cycle, restart, read | Data still present (power-failure safe) |
| Write 5 entries, delete #3, read  | Consecutive key names: dat1~dat4        |
| Input 4, restart, read            | Shows "No data"                         |

## 7. Common Issues

| Phenomenon                                   | Cause                                    | Solution                                                        |
|----------------------------------------------|------------------------------------------|-----------------------------------------------------------------|
| Data lost after restart                      | <code>prefs.end()</code> not called      | Must call <code>end()</code> after each write                   |
| Write fails/hangs                            | Opened for write in read-only mode       | Second parameter of <code>begin(ns, false)</code> must be false |
| Key not found but data exists                | Counter inconsistent with actual storage | Check if delete logic correctly updates count                   |
| Compilation error<br>Preferences.h not found | ESP32 board package not installed        | Install ESP32 Arduino Core in IDE                               |